

Goki

Audit



Presented by:

OtterSec contact@osec.io

Robert Chen notdeghost@osec.io

Kevin Chow kchow@osec.io

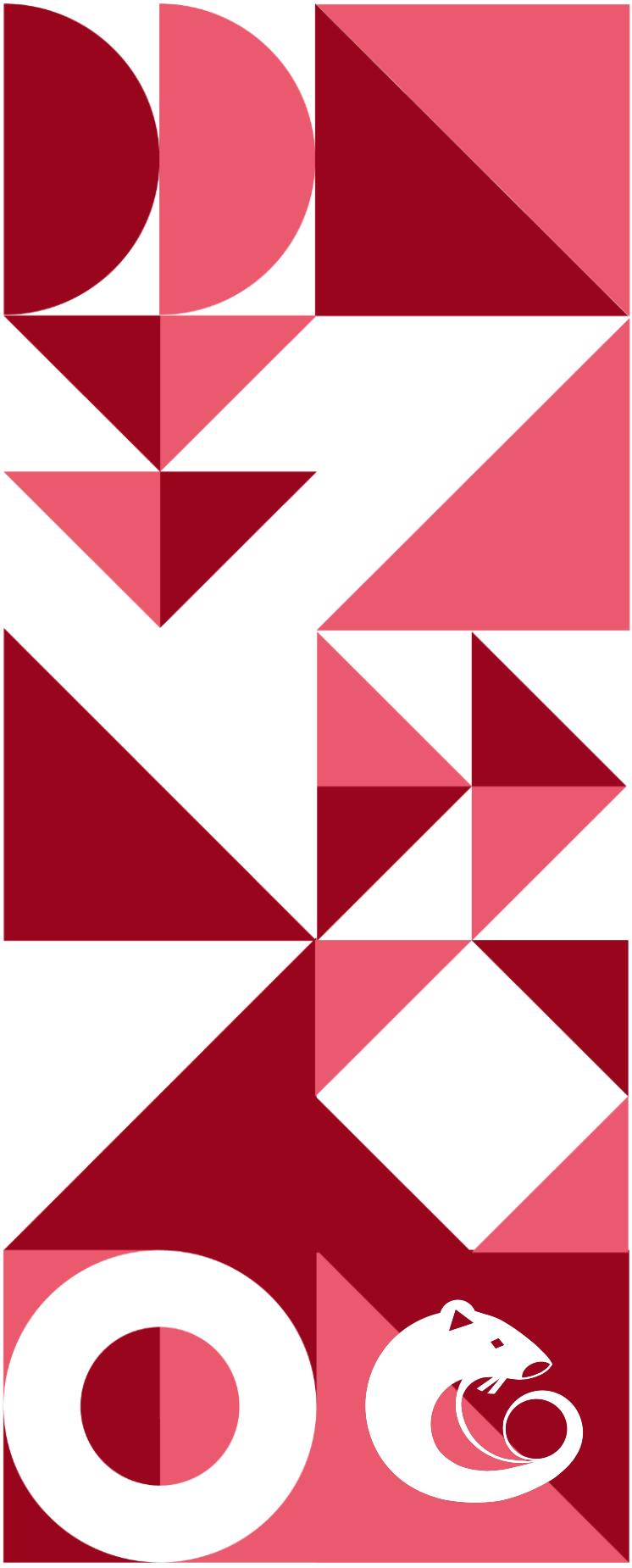


Table of Contents

Table of Contents **1**

01 | Executive Summary **2**
Overview 2
Key Findings 2

02 | Scope **3**

03 | Procedure **4**

04 | Findings **5**
Proof of Concepts 5

05 | Vulnerabilities **7**
OS-GOK-ADV-00 [Critical] [Resolved]: Unchecked Buffer Transactions 8

06 | General Findings **11**
OS-GOK-SUG-00: Smart Wallet Creation Thresholds 12
OS-GOK-SUG-01: Duplicate Owners 14
OS-GOK-SUG-02: Missing Owners Length Check 15
OS-GOK-SUG-03: Incorrect Discriminator Size 17
OS-GOK-SUG-04: Validator Account Checks 18

07 | Appendix **20**
Appendix A: Program Files 20
Appendix B: Implementation Security Checklist 21
Appendix C: Proof of Concepts 23
Appendix D: Vulnerability Rating Scale 24

© 2022 OtterSec LLC. All Rights Reserved.

01 | Executive Summary

Overview

Ship Capital engaged OtterSec to perform an assessment of the `goki` program prior to deployment. This assessment was conducted between April 26th and May 10th.

Critical vulnerabilities were communicated to the team prior to the delivery of the report to speed up remediation. After delivering our audit report, we worked closely with the Ship Capital team to streamline patches and confirm remediation.

Key Findings

The following is a summary of the major findings in this audit.

- 6 findings total
- 1 vulnerabilities which could lead to loss of funds
    - OS-GOK-ADV-00: Bypass of SmartWallet signature validation

As part of this audit, we also provided proof of concepts for each vulnerability to prove exploitability and enable simple regression testing. These scripts can be found at https://osec.io/pocs/goki. For a full list, see Appendix C.

© 2022 OtterSec LLC. All Rights Reserved.

02 | Scope

We received the program from Ship Capital and began the audit on April 26. The source code was delivered to us in a git repository at https://github.com/GokiProtocol/goki/. This audit was performed against commit 164d238.

Only the smart-wallet implementation was in scope.

A brief description of the program is as follows. A full list of program files and hashes can be found in Appendix A.

Name	Description
smart-wallet	The central program for managing multi-sig wallets. The functionality of this program includes: - Creating, voting, and executing transactions on behalf of the wallet - Invoking derived wallet signatures for multiple purposes - Initiating bundled transactions via an instruction buffer

© 2022 OtterSec LLC. All Rights Reserved.

03 | Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an onchain program. In other words, there is no way to steal tokens or deny service, ignoring any Solana specific quirks such as account ownership issues. An example of a design vulnerability would be an onchain oracle which could be manipulated by flash loans or large deposits.

On the other hand, auditing the implementation of the program requires a deep understanding of Solana's execution model. Some common implementation vulnerabilities include account ownership issues, arithmetic overflows, and rounding bugs. For a non-exhaustive list of security issues we check for, see Appendix B. We identified a critical issue here which would allow an attacker to sign for any smart wallet (OS-GOK-ADV-00).

Implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to get a comprehensive understanding of the program first. In our audits, we always approach any target in a team of two. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

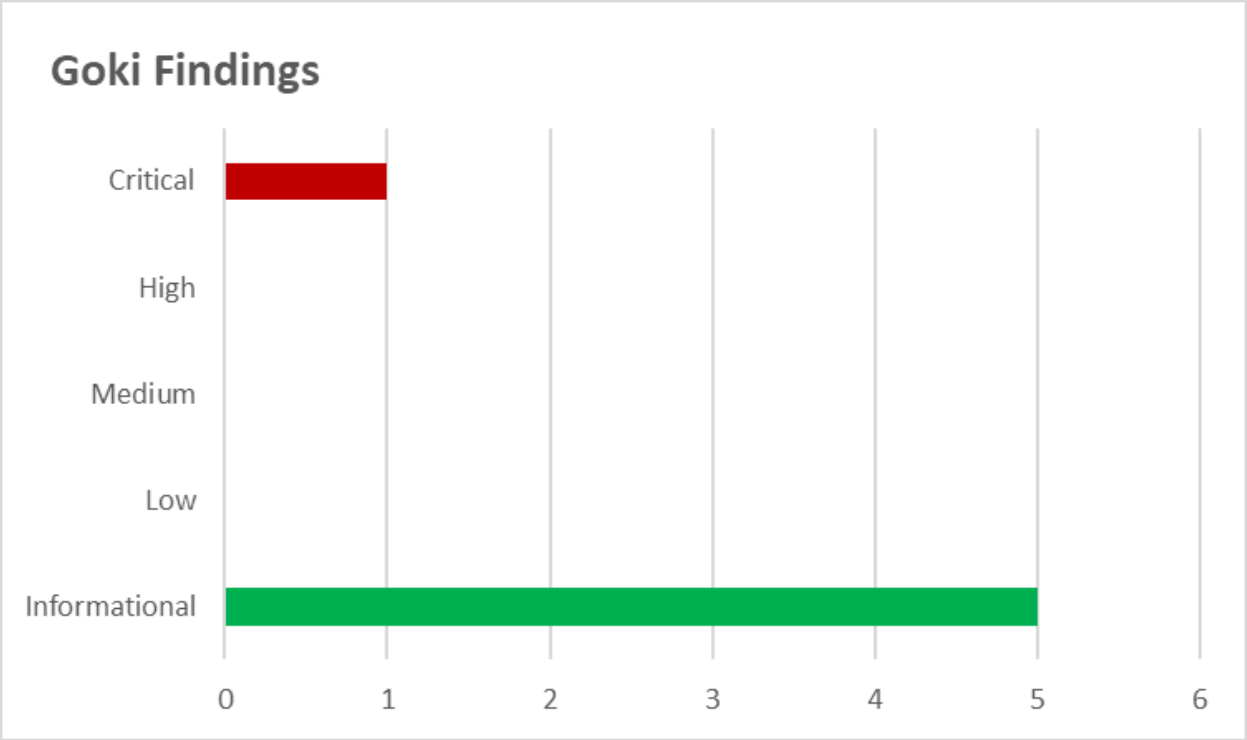
© 2022 OtterSec LLC. All Rights Reserved.

04 | Findings

Overall, we report 6 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings don't have an immediate impact but will help mitigate future vulnerabilities.

The below chart displays the findings by severity.



Proof of Concepts

For each vulnerability we created a proof of concept to enable easy regression testing. We recommend integrating these as part of a comprehensive test suite. The proof of concept directory structure can be found in Appendix C.

These proof of concepts can be found at https://osec.io/pocs/goki.

© 2022 OtterSec LLC. All Rights Reserved.

To run a POC:

```
./run.sh <directory name>
```

For example,

```
./run.sh os-gok-adv-00
```

Each proof of concept comes with its own patch file which modifies the existing test framework to demonstrate the relevant vulnerability. We also recommend integrating these patches into the test suite to prevent regressions.

© 2022 OtterSec LLC. All Rights Reserved.

05 | Vulnerabilities

Here we present a technical analysis of the vulnerabilities we identified during our audit of the `goki` program. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

Rating criterion can be found in Appendix D.

ID	Severity	Status	Description
OS-GOK-ADV-00	**Critical**	**Resolved**	Missing permission checks on InstructionBundle allow an attacker to sign for any smart wallet.

© 2022 OtterSec LLC. All Rights Reserved.

OS-GOK-ADV-00 [Critical] [Resolved]: Unchecked Buffer Transactions

Description

A lack of authentication on `InstructionBuffer` creation and execution allows an attacker to sign for any `SmartWallet`. An attacker could then change the `SmartWallet` owners, allowing them to completely take over the smart wallet. A less severe attack would be to lower the threshold of the `SmartWallet`.

The `InstructionBuffer` account allows a user to execute bundled instructions. These transactions are signed with the smart wallet seeds.

instructions/buffer_execute_bundle.rs

```
pub fn handler<'info>(
    ctx: Context<'_, '_, '_, 'info, ExecuteBufferBundle<'info>>,
    bundle_index: u8,
) -> Result<()> {
    let smart_wallet = &ctx.accounts.smart_wallet;
    let wallet_seeds: &[&[&[u8]]] = &[&[
        b"GokiSmartWallet" as &[u8],
        &smart_wallet.base.to_bytes(),
        &[smart_wallet.bump],
    ]];

...

    for ix in bundle.instructions.iter() {
        solana_program::program::invoke_signed(&(ix).into(),
ctx.remaining_accounts, wallet_seeds)?;
    }
    bundle.is_executed = true;
    buffer.set_bundle(b_index, &bundle)?;

    Ok(())
}
```

© 2022 OtterSec LLC. All Rights Reserved.

This `InstructionBuffer` initialization is unchecked - the `authority` and `executor` can be set on any smart_wallet. There are checks downstream on appending, finalizing, and executing instructions against the `authority` and `executor`, but these are moot because they can be set by anybody on initialization.

instructions/buffer_init.rs

```
/// Accounts for [smart_wallet::init_ix_buffer].
#[derive(Accounts)]
pub struct InitBuffer<'info> {
    #[account(zero)]
    /// The [InstructionBuffer].
    pub buffer: Account<'info, InstructionBuffer>,
    /// The [InstructionBuffer::smart_wallet].
    pub smart_wallet: Account<'info, SmartWallet>,
    /// CHECK: Arbitrary account allowed.
    pub authority: UncheckedAccount<'info>,
    /// CHECK: Arbitrary account allowed.
    pub executor: UncheckedAccount<'info>,
}
```

Proof of Concept

Consider the following steps

1. Create an instruction bundle for the victim smart wallet
2. Add a `set_owners` instruction with the attacker as the sole owner
3. Finalize the bundle
4. Execute the instruction
5. The attacker is now in control of the smart wallet

© 2022 OtterSec LLC. All Rights Reserved.

```
Owners BEFORE: [
  PublicKey {
    _bn: <BN: ef80b56da4cb3a0df9e163af6dcaa9f46673304e224043985e197fb9022ca3bc>
  }
]
Buffer Executor PublicKey {
  _bn: <BN: 432deb2a59a9c3eb5ab77d15457d51f996d04646d18ba89d5b6baf9017f9790>
}
Buffer Authority PublicKey {
  _bn: <BN: 432deb2a59a9c3eb5ab77d15457d51f996d04646d18ba89d5b6baf9017f9790>
}
Owners AFTER: [
  PublicKey {
    _bn: <BN: 432deb2a59a9c3eb5ab77d15457d51f996d04646d18ba89d5b6baf9017f9790>
  }
]
```

Remediation

The `InstructionBuffer` initialization instruction needs to be validated against the owners of the smart wallet.

Patch

Patched in c3bfdf2, by removing the instruction buffer feature.

© 2022 OtterSec LLC. All Rights Reserved.

06 | General Findings

Here we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they do represent antipatterns and could introduce a vulnerability in the future.

ID	Description
OS-GOK-SUG-00	Enforce sane thresholds on smart wallet creation
OS-GOK-SUG-01	Support or Deny duplicate owners on smart wallets
OS-GOK-SUG-02	Validate length of owners parameter while updating owners
OS-GOK-SUG-03	Incorrect discriminator size used on smart wallet creation
OS-GOK-SUG-04	Account checks performed inconsistently with validator

© 2022 OtterSec LLC. All Rights Reserved.

OS-GOK-SUG-00: Smart Wallet Creation Thresholds

Description

SmartWallet initialization fails to ensure that the signature threshold is less than or equal to the number of owners. This can lead to wallets that are dead upon creation, making it impossible to sign transactions if the threshold is accidentally created too high.

src/lib.rs

```
    /// Initializes a new [SmartWallet] account with a set of owners
and a threshold.
    #[access_control(ctx.accounts.validate())]
    pub fn create_smart_wallet(
        ctx: Context<CreateSmartWallet>,
        _bump: u8,
        max_owners: u8,
        owners: Vec<Pubkey>,
        threshold: u64,
        minimum_delay: i64,
    ) -> Result<()> {
        invariant!(minimum_delay >= 0, "delay must be positive");
        invariant!(minimum_delay < MAX_DELAY_SECONDS, DelayTooHigh);

        invariant!((max_owners as usize) >= owners.len(),
"max_owners");

        let smart_wallet = &mut ctx.accounts.smart_wallet;
        smart_wallet.base = ctx.accounts.base.key();
        smart_wallet.bump =
*unwrap_int!(ctx.bumps.get("smart_wallet"));

        smart_wallet.threshold = threshold;
        smart_wallet.minimum_delay = minimum_delay;
        smart_wallet.grace_period = DEFAULT_GRACE_PERIOD;

        smart_wallet.owner_set_seqno = 0;
        smart_wallet.num_transactions = 0;

        smart_wallet.owners = owners.clone();
```

Note that this invariant is explicitly enforced when changing the threshold.

© 2022 OtterSec LLC. All Rights Reserved.

src/lib.rs

```
    /// Changes the execution threshold of the smart_wallet. The only way
this can be
    /// invoked is via a recursive call from execute_transaction ->
    /// change_threshold.
    #[access_control(ctx.accounts.validate())]
    pub fn change_threshold(ctx: Context<Auth>, threshold: u64) ->
Result<()> {
        invariant!(
            threshold <= ctx.accounts.smart_wallet.owners.len() as u64,
            InvalidThreshold
        );
        let smart_wallet = &mut ctx.accounts.smart_wallet;
        smart_wallet.threshold = threshold;

        emit!(WalletChangeThresholdEvent {
            smart_wallet: ctx.accounts.smart_wallet.key(),
            threshold,
            timestamp: Clock::get()?.unix_timestamp
        });
        Ok(())
    }
```

Remediation

Add an additional invariant to ensure that the threshold is sane when creating a smart wallet.

```
        invariant!(
            threshold <= ctx.accounts.smart_wallet.owners.len() as u64,
            InvalidThreshold
        );
```

© 2022 OtterSec LLC. All Rights Reserved.

OS-GOK-SUG-01: Duplicate Owners

Description

Duplicate owners can be set on smart wallets. However, this is not supported because only the first index can be set of the owner.

src/state.rs

```
/// Gets the index of the key in the owners Vec, or None
    pub fn owner_index_opt(&self, key: Pubkey) -> Option<usize> {
        self.owners.iter().position(|a| *a == key)
    }

    /// Gets the index of the key in the owners Vec, or error
    pub fn try_owner_index(&self, key: Pubkey) -> Result<usize> {
        Ok(unwrap_opt!(self.owner_index_opt(key), InvalidOwner))
    }
```

Remediation

Either check owners are unique or support the wallet having duplicate owners (duplicate owners would have a higher vote weight)

© 2022 OtterSec LLC. All Rights Reserved.

OS-GOK-SUG-02: Missing Owners Length Check

Description

The `set_owners` instruction does not validate the length of the `owners` parameter. This can cause the program to panic if the length is larger than `max_owners`, as specified during the creation of the smart wallet.

src/lib.rs

```
    #[access_control(ctx.accounts.validate())]
    pub fn set_owners(ctx: Context<Auth>, owners: Vec<Pubkey>) ->
Result<()> {
        let smart_wallet = &mut ctx.accounts.smart_wallet;
        if (owners.len() as u64) < smart_wallet.threshold {
            smart_wallet.threshold = owners.len() as u64;
        }

        smart_wallet.owners = owners.clone();
```

Note that this invariant is enforced in the `create_smart_wallet` instruction [0].

src/lib.rs

```
    #[access_control(ctx.accounts.validate())]
    pub fn create_smart_wallet(
        ctx: Context<CreateSmartWallet>,
        _bump: u8,
        max_owners: u8,
        owners: Vec<Pubkey>,
        threshold: u64,
        minimum_delay: i64,
    ) -> Result<()> {
        invariant!(minimum_delay >= 0, "delay must be positive");
        invariant!(minimum_delay < MAX_DELAY_SECONDS, DelayTooHigh);

        invariant!((max_owners as usize) >= owners.len(),
"max_owners"); // [0]
```

© 2022 OtterSec LLC. All Rights Reserved.

Remediation

Store `max_owners` in the `SmartWallet` structure and add an invariant as present in `create_smart_wallet`.

```
        invariant!((max_owners as usize) >= owners.len(),
"max_owners");
```

© 2022 OtterSec LLC. All Rights Reserved.

OS-GOK-SUG-03: Incorrect Discriminator Size

Description

The discriminator size used while creating the `SmartWallet` account is incorrect [0]. The actual size of Anchor's discriminator is 8 bytes.

src/state.rs

```
impl SmartWallet {
    /// Computes the space a [SmartWallet] uses.
    pub fn space(max_owners: u8) -> usize {
        4 // Anchor discriminator [0]
            + std::mem::size_of::<SmartWallet>()
```

This is not a problem in the current state of the program, due to the reserved space present in the `SmartWallet` structure [1]. However, it is recommended to update the discriminator size to avoid any future implications.

src/state.rs

```
#[derive(Default, Debug, PartialEq)]
pub struct SmartWallet {
    [...]
    pub reserved: [u64; 16], // [1]
}
```

Remediation

Update the Anchor discriminator size to 8 bytes.

© 2022 OtterSec LLC. All Rights Reserved.

OS-GOK-SUG-04: Validator Account Checks

Description

While reviewing the program, it was discovered that account checks were performed in an inconsistent manner. Some of the account checks were performed with the validator, while others with anchor attributes. It is recommended to completely separate account validation from the program's logic. For this, anchor implements a list of account constraints which can be utilized.

Moreover, it was found that some checks were performed twice [0] [1]. The following code snippets show an example of this case.

src/approve.rs

```
/// Accounts for [smart_wallet::approve].
#[derive(Accounts)]
pub struct Approve<'info> {
    /// The [SmartWallet].
    pub smart_wallet: Account<'info, SmartWallet>,
    /// The [Transaction].
    #[account(mut, has_one = smart_wallet)] // [0]
    pub transaction: Account<'info, Transaction>,
    /// One of the smart_wallet owners. Checked in the handler.
    pub owner: Signer<'info>,
}
```

src/approve.rs

```
/// This validator is used for both approve and unapprove.
impl<'info> Validate<'info> for Approve<'info> {
    fn validate(&self) -> Result<()> {
        assert_keys_eq!(self.smart_wallet,
self.transaction.smart_wallet); // [1]

        invariant!(
            self.smart_wallet.owner_set_seqno ==
self.transaction.owner_set_seqno,
            OwnerSetChanged
        );
```

© 2022 OtterSec LLC. All Rights Reserved.

```
        invariant!(self.transaction.executed_at == -1,
AlreadyExecuted);
```

This general coding style can make it harder to reason about the checks.

Remediation

Consolidate the security checks in one place. For example, always use anchor attributes for account validation. For more information, refer to the code snippet below.

```
use crate::ErrorCode;
/// Accounts for [smart_wallet::approve].
#[derive(Accounts)]
pub struct Approve<'info> {
    /// The [SmartWallet].
    #[account(
        constraint = smart_wallet.owner_set_seqno ==
transaction.owner_set_seqno @ErrorCode::OwnerSetChanged
    )]
    pub smart_wallet: Account<'info, SmartWallet>,
    /// The [Transaction].
    #[account(mut, has_one = smart_wallet,
        constraint = transaction.executed_at == -1
@ErrorCode::AlreadyExecuted
    )]
    pub transaction: Account<'info, Transaction>,
    /// One of the smart_wallet owners. Checked in the handler.
    pub owner: Signer<'info>,
}
```

© 2022 OtterSec LLC. All Rights Reserved.

07 | Appendix

Appendix A: Program Files

Below are the files in scope for this audit and their corresponding sha256 hashes.

```
Cargo.toml                                dda346447bca89cc6bd78d2de1775bbd
README.md                                 306d08f212366e1aec60f182885c5dbc
Xargo.toml                                815f2dfb6197712a703a8e1f75b03c69
src
  events.rs                               30280f8db168b0d8f57c36ddbe89b195
  lib.rs                                  dbf3ef220873d26feabbbd06b5640e2f
  state.rs                                e4ae6284e72b20e744f5eb86ece1241f
  validators.rs                           b95d218b51129067926c4cdce4583527
  instructions
    approve.rs                            d0c4149eb4fc4273bc55b987b5fc65ba
    buffer_append_ix.rs                   d2c2a0f6796b235bff4c7d1738216218
    buffer_close.rs                       639107e2e31c8e694c949b5d2f40dfc4
    buffer_execute_bundle.rs              d9a42d49eb654956eb00e04638781474
    buffer_finalize.rs                    b636359e9d5f6c9cc1560c4977449c76
    buffer_init.rs                        42b7583d74d3ba5d62bbeb0b3d46c3df
    mod.rs                                19ca4cdd8ba82b16eddf2b5f1688c1b7
    unapprove.rs                          f5d4014ea3cee16661eeee8f6ff6312f
```

© 2022 OtterSec LLC. All Rights Reserved.

Appendix B: Implementation Security Checklist

Unsafe arithmetic

Integer underflows or overflows	Unconstrained input sizes could lead to integer over or underflows, causing potentially unexpected behavior. Ensure that for unchecked arithmetic, all integers are properly bounded.
Rounding	Rounding should always be done against the user to avoid potentially exploitable off-by-one vulnerabilities.
Conversions	Rust `as` conversions can cause truncation if the source value does not fit into the destination type. While this is not undefined behavior, such truncation could still lead to unexpected behavior by the program.

Account Security

Account Ownership	Account ownership should be properly checked to avoid type confusion attacks. For Anchor, the safety of unchecked accounts should be clearly justified and immediately obvious.
Accounts	For non-Anchor programs, the type of the account should be explicitly validated to avoid type confusion attacks.
Signer Checks	Privileged operations should ensure that the operation is signed by the correct accounts.
PDA Seeds	PDA seeds are uniquely chosen to differentiate between different object classes, avoiding collision.

Input Validation

Timestamps	Timestamp inputs should be properly validated against the current clock time. Timestamps which are meant to be in the future should be explicitly validated so.
Numbers	Sane limits should be put on numerical input data to mitigate the risk of unexpected over and underflows. Input data should be constrained to the smallest size type possible, and upcasted for unchecked arithmetic.
Strings	Strings should have sane size restrictions to prevent denial of service conditions
Internal State	If there is internal state, ensure that there is explicit validation on the

© 2022 OtterSec LLC. All Rights Reserved.

	input account’s state before engaging in any state transitions. For example, only open accounts should be eligible for closing.

Miscellaneous

Libraries	Out of date libraries should not include any publicly disclosed vulnerabilities
Clippy	`cargo clippy` is an effective linter to detect potential anti-practices.

© 2022 OtterSec LLC. All Rights Reserved.

Appendix C: Proof of Concepts

Below are the provided proof of concept files and their sha256 hashes.

```
run.sh                                  56ce0c258f1711a68985a2dbe92cb01c
os-gok-adv-00
  patch                                 e2331a77a853b972a57df0396eba8be6
  run.sh                                f4bc0f924aeb699a36b942f893bc9169
```

© 2022 OtterSec LLC. All Rights Reserved.

Appendix D: Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings can be found in the General Findings section.

Critical	Vulnerabilities which **immediately** lead to loss of user funds with minimal preconditions Examples: - Misconfigured authority/token account validation - Rounding errors on token transfers
High	Vulnerabilities which **could** lead to loss of user funds but are potentially difficult to exploit. Examples: - Loss of funds requiring specific victim interactions - Exploitation involving high capital requirement with respect to payout
Medium	Vulnerabilities which could lead to denial of service scenarios or degraded usability. Examples: - Malicious input cause computation limit exhaustion - Forced exceptions preventing normal use
Low	Low probability vulnerabilities which could still be exploitable but require extenuating circumstances or undue risk. Examples: - Oracle manipulation with large capital requirements and multiple transactions
Informational	Best practices to mitigate future security risks. These are classified as *general findings*. Examples: - Explicit assertion of critical internal invariants - Improved input validation - Uncaught Rust errors (vector out of bounds indexing)

© 2022 OtterSec LLC. All Rights Reserved.